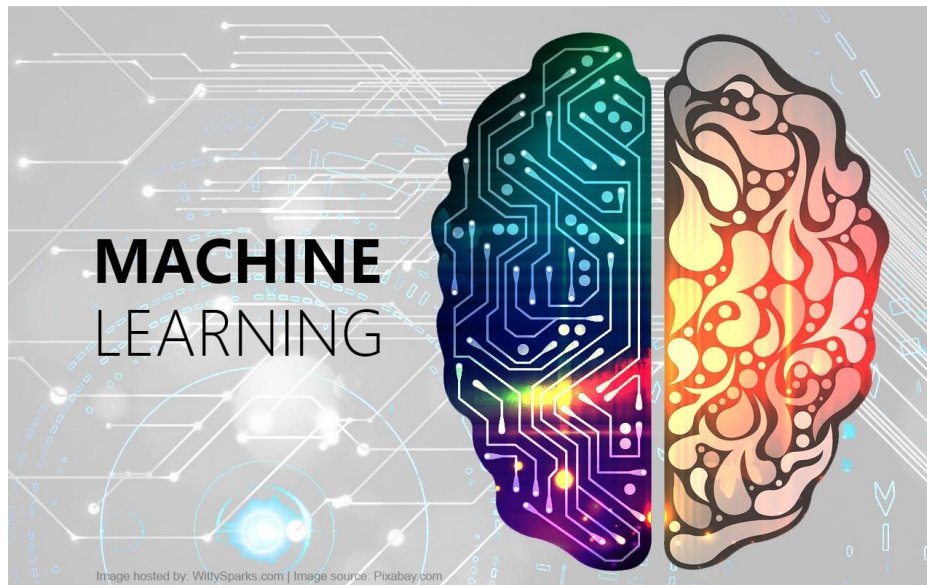


Machine learning

Lecture 3 - Dive into ML

07.04.2024

Toker Gilat



Introduction

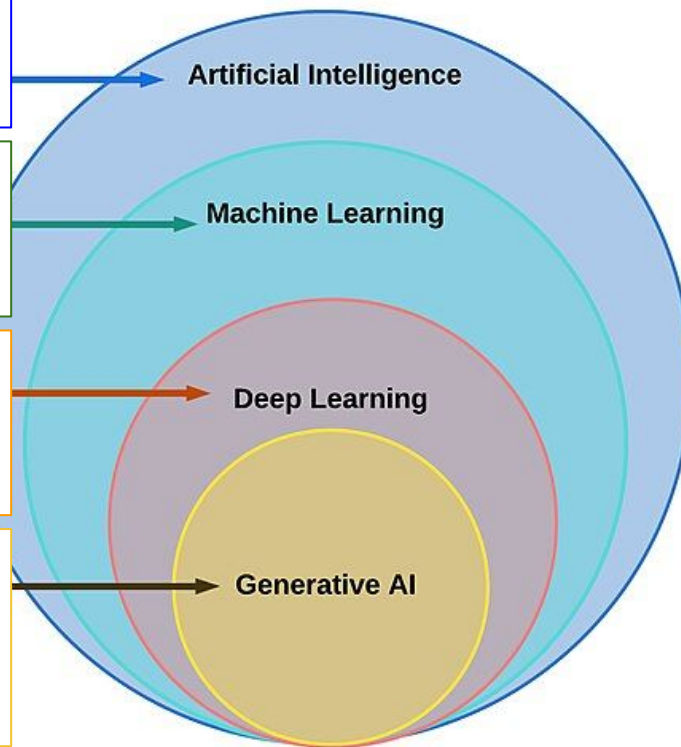
What is machine learning?

Artificial Intelligence - AI involves techniques that equip computers to emulate human behavior, enabling them to learn, make decisions, recognize patterns, and solve complex problems.

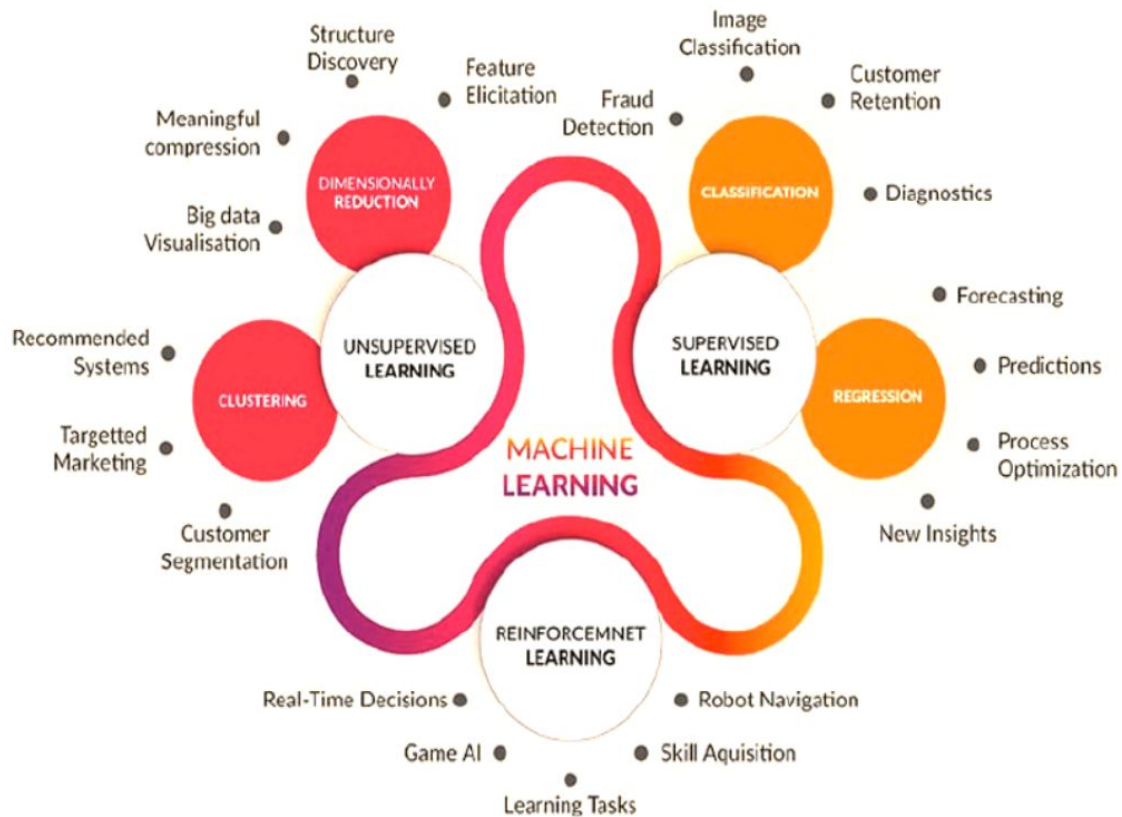
Machine Learning - ML is a subset of AI, uses advanced algorithms to detect patterns in large data sets, allowing machines to learn and adapt. ML algorithms use **supervised or unsupervised learning methods**.

Deep Learning - DL is a subset of ML which uses neural networks for in-depth data processing and analytical tasks. DL leverages multiple layers of artificial neural networks to extract high-level features, simulating the human brains perceive.

Generative AI - Generative AI is a subset of DL models that generates content like text, images, or code based on provided input. Trained on vast data sets, these models detect patterns and create outputs without explicit instruction, using a mix of supervised and unsupervised learning.



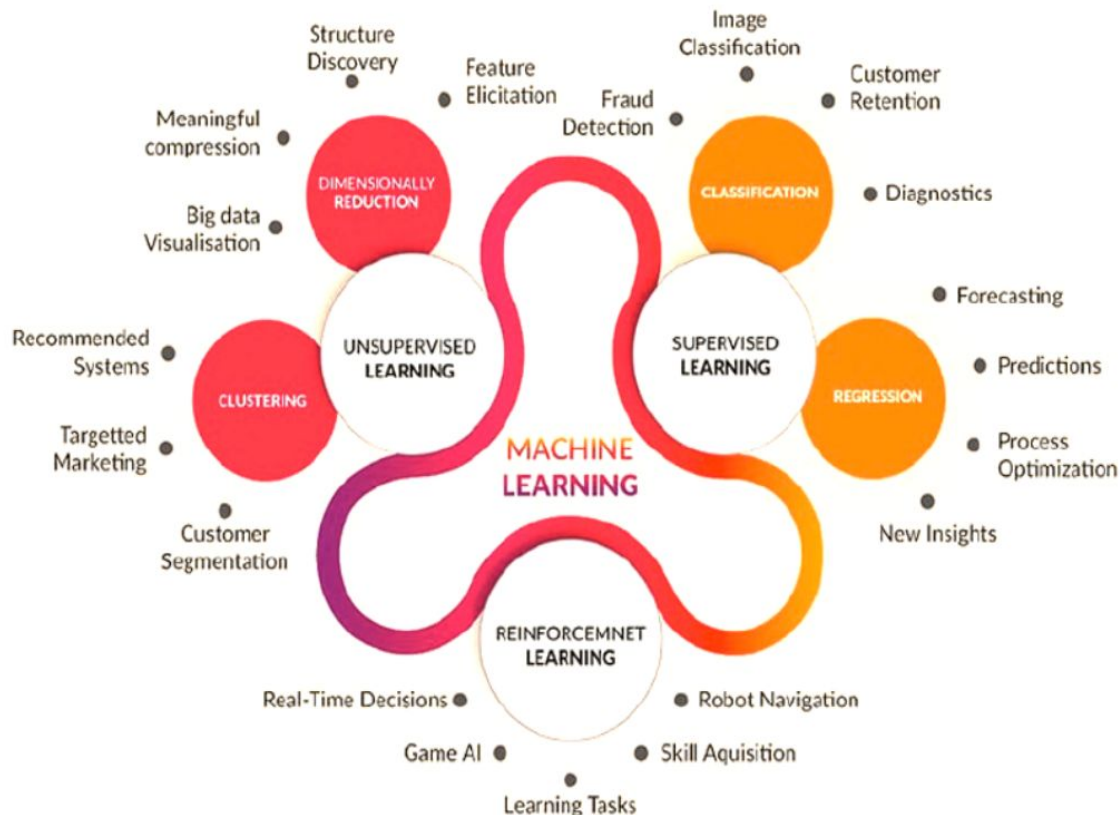
What is machine learning?



What is machine learning?

Supervised Learning:

Supervised machine learning uses labeled data, where each input has a corresponding output. The model learns the relationship between inputs and outputs to predict outcomes for new data.



Supervised learning

Learning from examples:

X : predictors/features

y : target(s)/label(s)

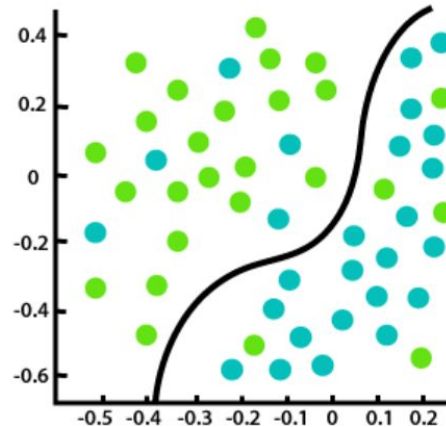
Goal: finding a function $f(X) \rightarrow y$

Train: $\text{train}(X, y) \rightarrow \text{"model"}$

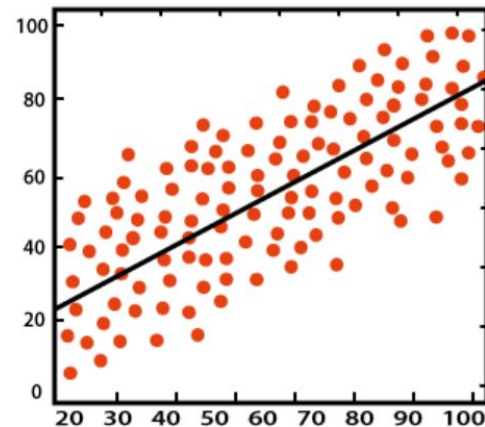
Predict: y_{red}

Evaluate: $\text{eval}(y_{\text{rue}}, y_{\text{red}}) \rightarrow \text{metrics}$

Types: Regression, Classification, and more...



Classification

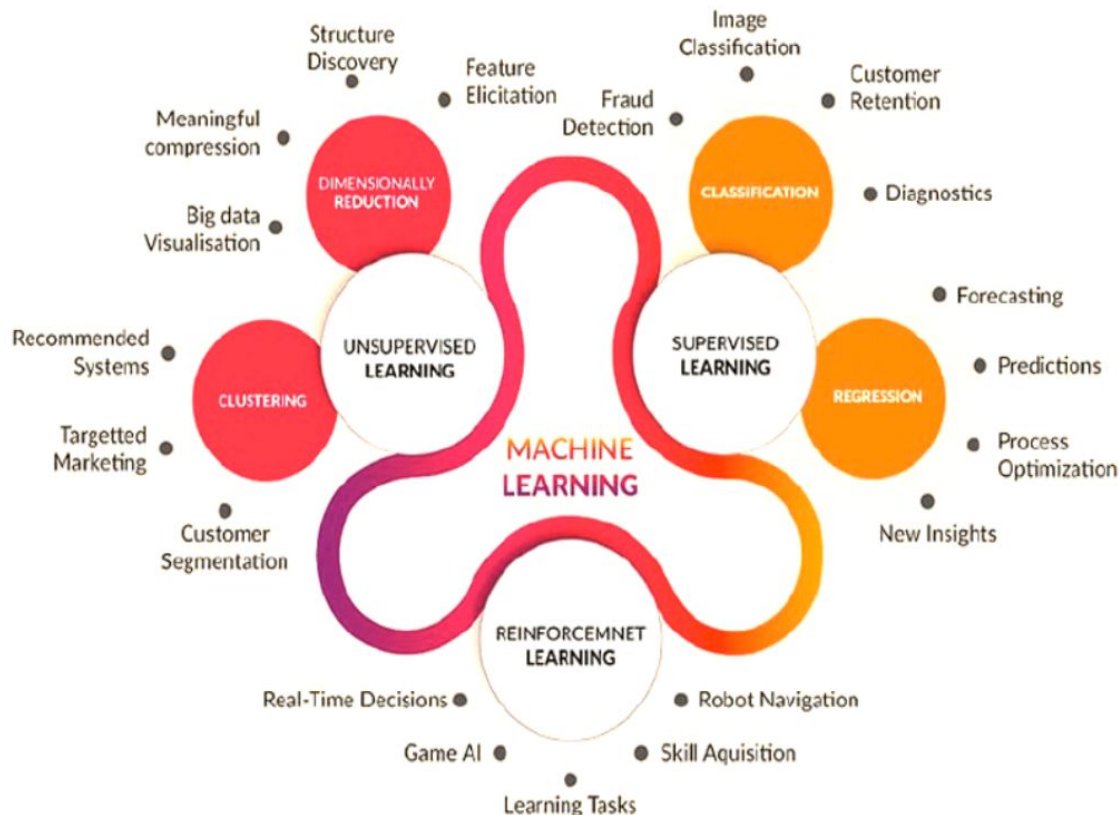


Regression

What is machine learning?

Unsupervised Learning:

Unsupervised machine learning works with unlabeled data, identifying patterns or clusters within the data without predefined outputs. It's often used for grouping similar items or discovering hidden structures.



Unsupervised learning

Learning patterns from unlabeled examples:

X: predictors/features

y: target(s)/label(s)

Goal: finding a function $f(X) \rightarrow y$

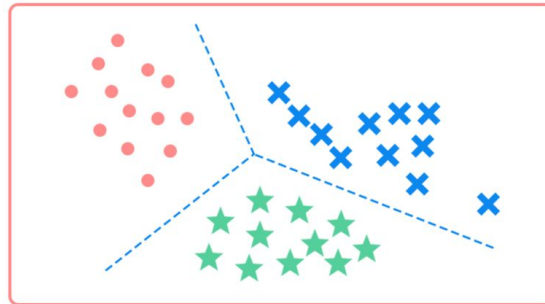
Train: $\text{train}(X, y) \rightarrow \text{"model"}$

Predict: y_{red}

Evaluate: $\text{eval}(y_{\text{rue}}, y_{\text{red}}) \rightarrow \text{metrics}$

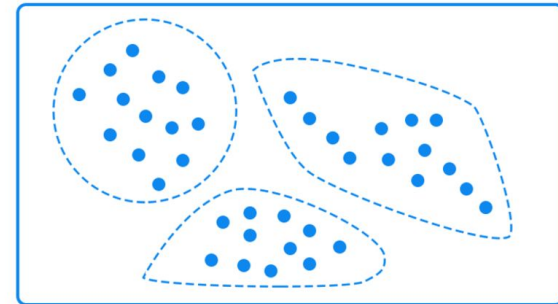
Types: Clustering, Anomaly Detection, and more...

Classification



Supervised learning

Clustering



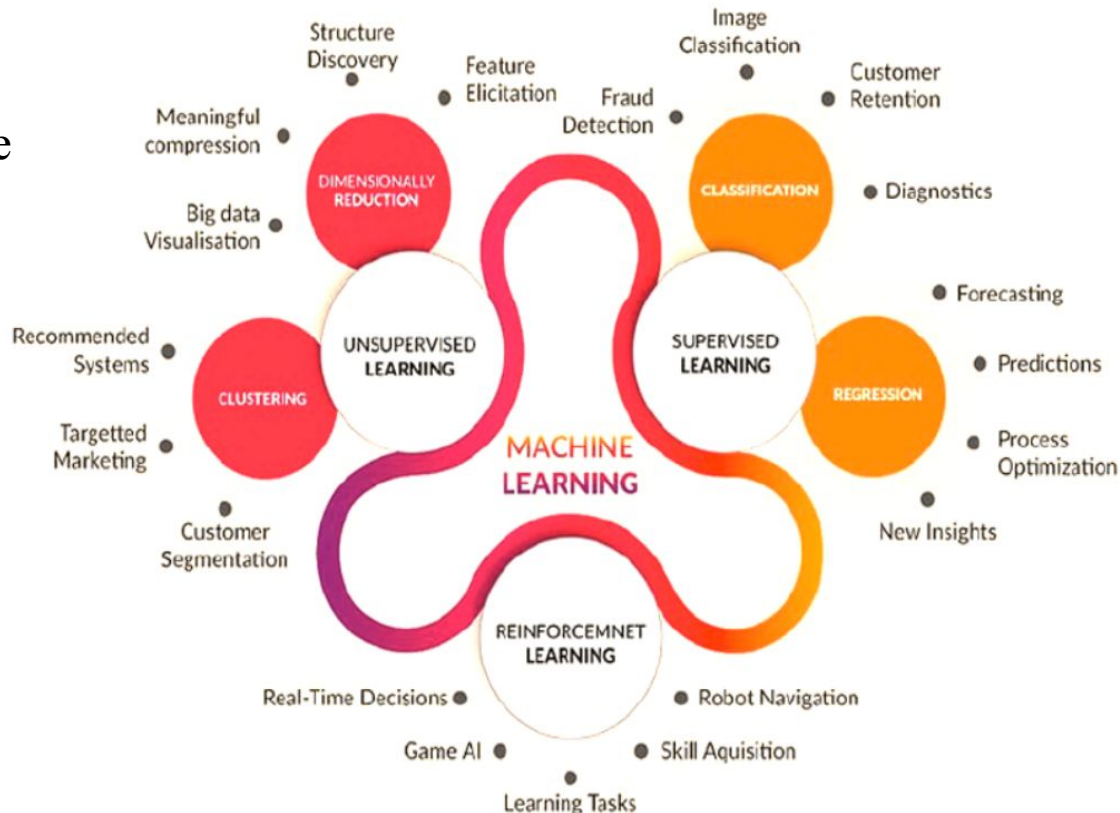
Unsupervised learning

What is machine learning?

Reinforcement Learning:

a type of machine learning where a model learns by interacting with an environment.

The model receives feedback through rewards or penalties based on its actions and adjusts its behavior to maximize cumulative rewards over time.



Reinforcement learning

Learning by interacting with an environment:

S: State (current situation or observation in the environment)

A: Action (decision taken by the agent based on the current state)

R: Reward (feedback signal indicating the success of the action)

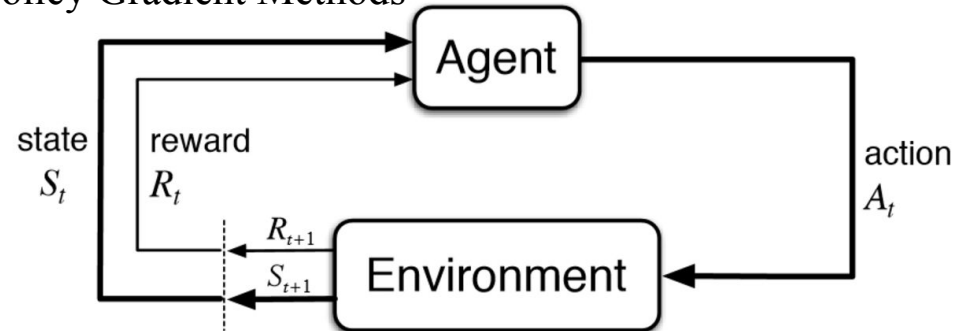
Goal: Maximizing cumulative reward over time by learning a policy $\pi(S) \rightarrow A$

Train: Iteratively update the policy $\pi(S)$ based on state, action, and reward using experience

Act: $A_{\text{red}} = \pi(S)$ (determine the next action based on the current policy)

Evaluate: Measure the agent's performance using cumulative or episodic reward

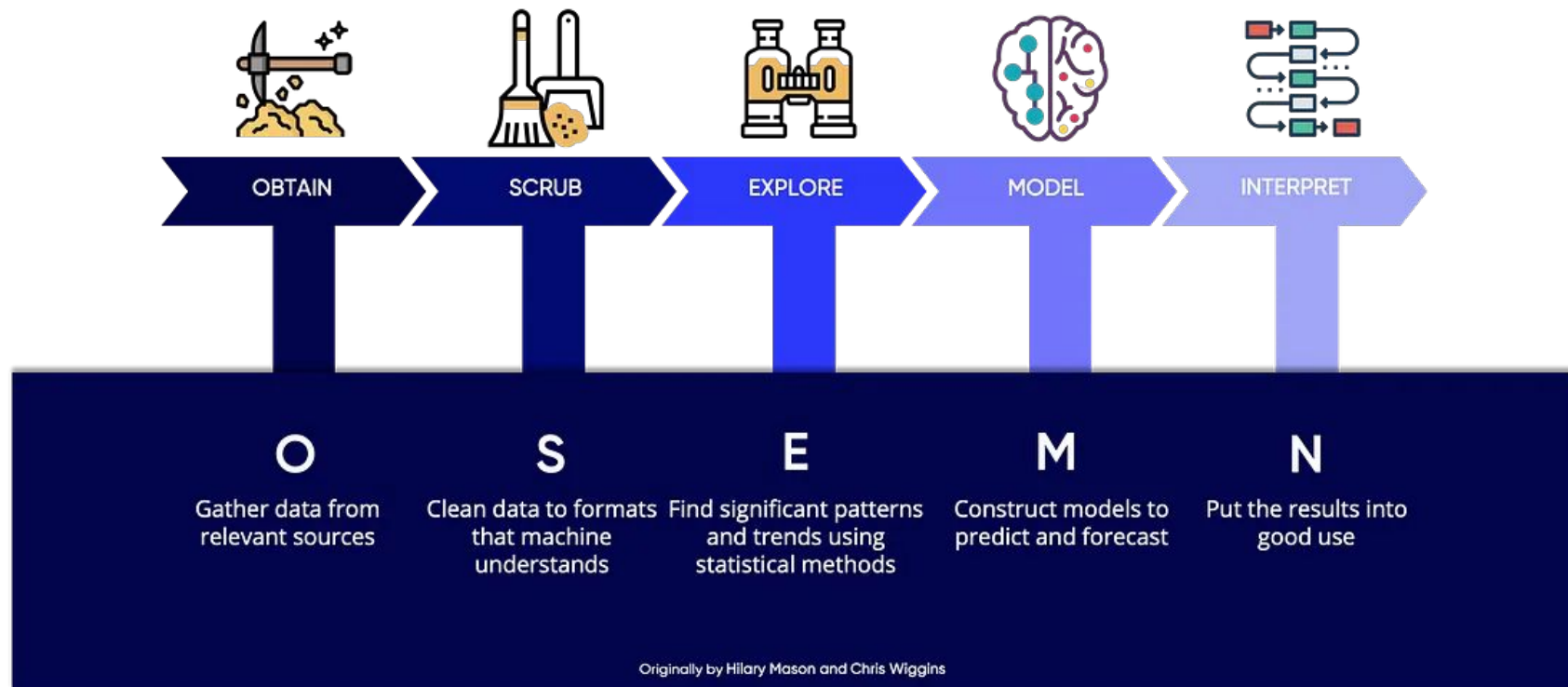
Types: Q-Learning, Deep Q-Learning (DQN), Policy Gradient Methods



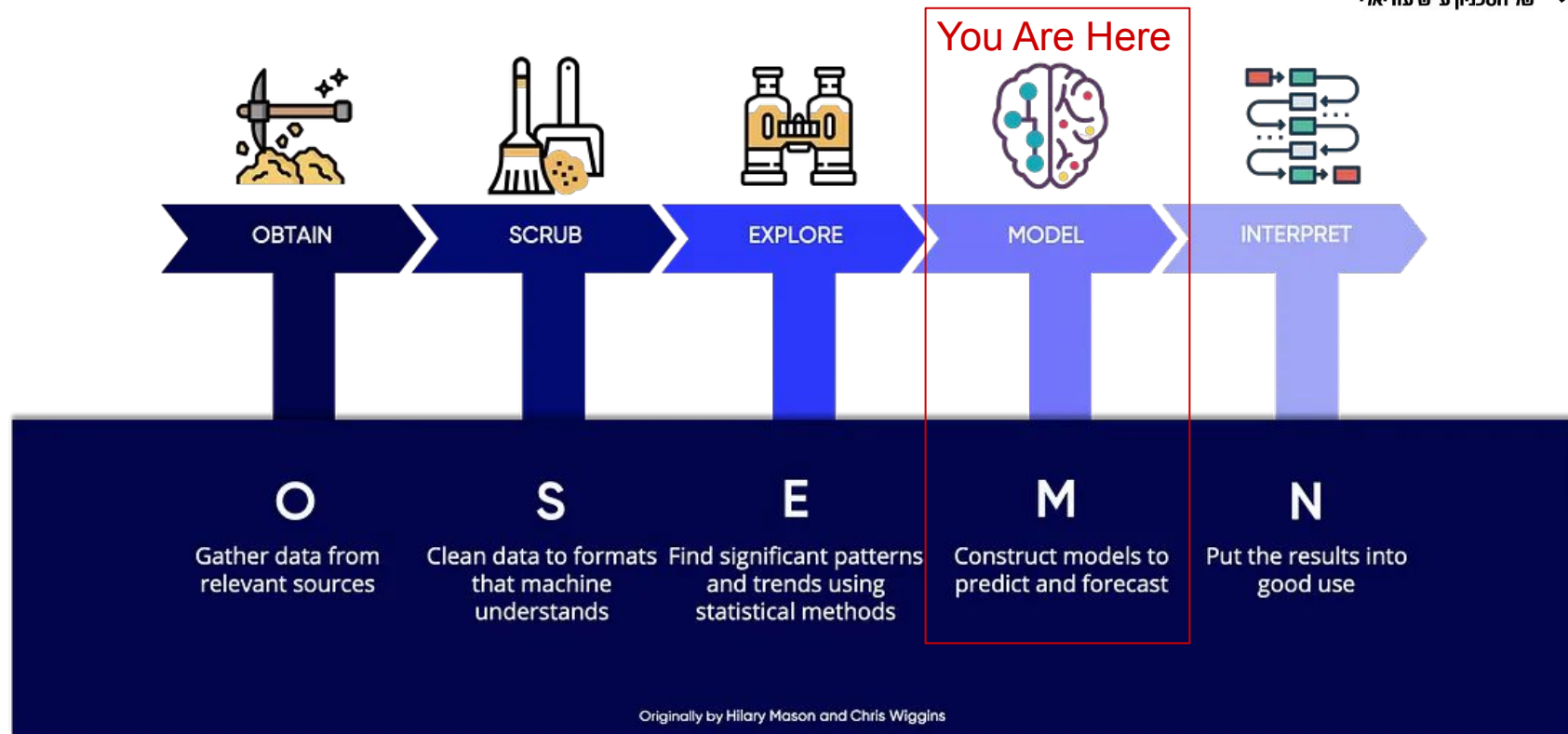
Supervised vs. Unsupervised vs. reinforcement learning

Aspect	Supervised	Unsupervised	Reinforcement
Data	Labeled	Unlabeled	Feedback from interaction
Goal	Predict labels or values	Discover patterns	Maximize cumulative rewards
Examples	Spam detection, Price prediction	Customer segmentation	Game-playing, Robotics

Data Science Process



Data Science Process



Methodology

1. problem definition -

Define as clearly as possible before jumping to code (or conclusions)

Task

- Understand the Domain
- What do we want to achieve? (“business” problem)
- How do we define success? (evaluation)
- Other considerations? (application, resources, human interaction etc.)

Data

- What’s the story? Where, how, why, who...
- Source, type, size, structure, availability, validity, missing data...
- Labels (if any) - understand the labeling scheme/process?
- Learning (train) & Inference (test)

Methodology

2. Solution development -

Hypothesis → method (e.g. model)

- Correlation between X and y
- But also: “similar users have similar taste” (e.g. recommendation systems)

Define Evaluation method

- Define appropriate evaluation metrics
- Evaluate w.r.t different aspects (solution is “good” → accuracy, coverage, etc.)

Analysis of ML models

- Where/when/in which cases do we perform well? Underperform? Can we improve?
How? (many methods and tricks)

Methodology

3. Towards a product

Presentation

- How can we best describe and visualize the methods and the results?
- Can we provide an explanation? (explainability vs. interpretability)
- Can we provide risk estimation? (confidence, model calibration...)

Deployment

- Productization (models/data/etc. → product, service...)
- Monitoring & maintenance (e.g. continuous learning)
- Many more “real-life” challenges...

Iterate, refine, extend...

What else? Depends on the project/company etc. Whatever you define

Linear regression - Evaluation

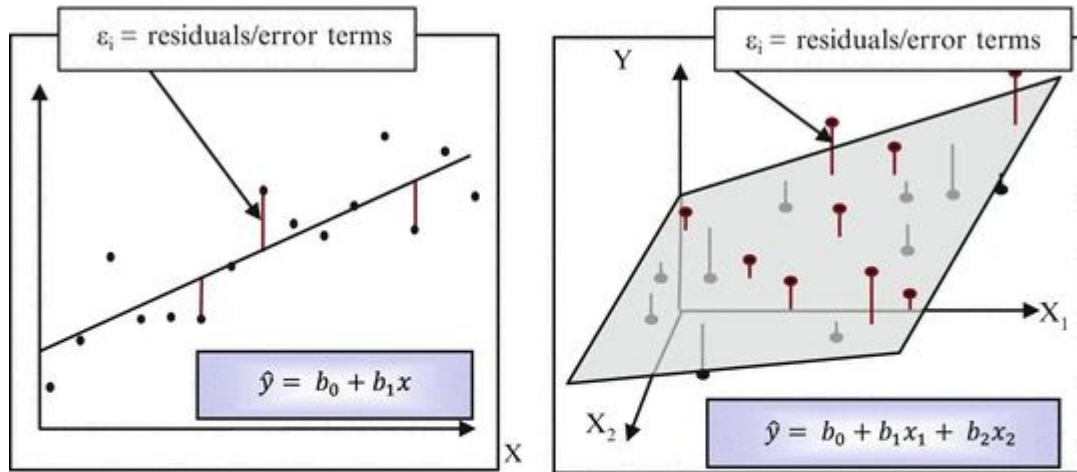
Linear Regression - reminder

The goal of the algorithm is to find the best Fit Line equation that can predict the values based on the independent variables.

There are two main types of linear regression:

Simple Linear Regression: $y = \beta_0 + \beta_1 X$

Multiple Linear Regression: $y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots \dots \dots \beta_n X_n$



Linear Regression - evaluation

Essential step in any machine learning model is to evaluate the model.

Mean Squared Error - Mean Squared Error represents the average of the squared difference between the original and predicted values in the data set. It measures the variance of the residuals

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

* Measures the average squared difference, Penalizes larger errors more heavily.

Linear Regression - evaluation

Essential step in any machine learning model is to evaluate the model.

Mean Squared Error - Mean Squared Error represents the average of the squared difference between the original and predicted values in the data set. It measures the variance of the residuals

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

Mean absolute error - The Mean absolute error represents the average of the absolute difference between the actual and predicted values in the dataset. It measures the average of the residuals in the dataset.

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

* Treats all errors equally, regardless of size. When outliers need less influence on the metric.

Linear Regression - evaluation

Essential step in any machine learning model is to evaluate the model.

Mean Squared Error - Mean Squared Error represents the average of the squared difference between the original and predicted values in the data set. It measures the variance of the residuals

$$MSE = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

Mean absolute error - The Mean absolute error represents the average of the absolute difference between the actual and predicted values in the dataset. It measures the average of the residuals in the dataset.

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Root Mean Squared Error - is the square root of Mean Squared error. It measures the standard deviation of residuals

$$RMSE = \sqrt{\frac{\sum_{i=1}^n (\hat{y}_i - y_i)^2}{n}}$$

* Provides an error metric in the same unit as the target variable, making it easier to interpret.

Linear Regression - evaluation

Essential step in any machine learning model is to evaluate the model.

R-Squared - represents the proportion of the variance in the dependent variable which is explained by the linear regression model. It is a scale-free score i.e. irrespective of the values being small or large, the value of R square will be less than one.

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

* Indicates the proportion of variance in the target variable explained by the model.
Values range from 0 to 1 (closer to 1 is better).

Linear Regression - evaluation

Essential step in any machine learning model is to evaluate the model.

R-Squared - represents the proportion of the variance in the dependent variable which is explained by the linear regression model. It is a scale-free score i.e. irrespective of the values being small or large, the value of R square will be less than one.

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

Adjusted R-Squared - This metric takes into account the model complexity as well, punishing models which have many dependent variables which does not contribute to the explainability of the dataset.

$$R_{adj}^2 = 1 - (1 - R^2) \cdot \frac{n - 1}{n - p - 1}$$

n: Number of observations.

p: Number of predictors.

* Penalizes adding irrelevant predictors to avoid overfitting.

Linear Regression - evaluation

Essential step in any machine learning model is to evaluate the model.

R-Squared - represents the proportion of the variance in the dependent variable which is explained by the linear regression model. It is a scale-free score i.e. irrespective of the values being small or large, the value of R square will be less than one.

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

Adjusted R-Squared - This metric takes into account the model complexity as well, punishing models which have many dependent variables which does not contribute to the explainability of the dataset.

$$R_{adj}^2 = 1 - (1 - R^2) \cdot \frac{n - 1}{n - p - 1}$$

n : Number of observations.

p : Number of predictors.

* Penalizes adding irrelevant predictors to avoid overfitting.

And much more ...

https://scikit-learn.org/stable/modules/model_evaluation.html#regression-metrics

Hands on !



Open Google colab notebook -
`Linear_Regression.ipynb`

Feature Importance

Feature Importance in Linear Regress

$$y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots \beta_n X_n$$

Feature importance in linear regression indicates how much each independent variable (x) contributes to predicting the dependent variable (y).

- It is derived from the **coefficients (β)** of the model equation
- Higher coefficient values (**in absolute terms**) imply that the feature has a larger impact on the target variable.
- Coefficients depend on the scale of the features. Solution: **Scale features** to standardize their ranges

Feature Importance in Linear Regress

Common Normalization Techniques:

1. Standardization (Z-Score Normalization):

Definition: Transform data to have a mean of 0 and a standard deviation of 1.

Formula:

$$X_{\text{scaled}} = \frac{X - \mu}{\sigma}$$

X : Original feature value.

μ : Mean of the feature.

σ : Standard deviation of the feature.

2. Min-Max Scaling (Normalization)

Definition: Scale data to a fixed range, typically [0, 1].

Formula:

$$X_{\text{scaled}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

* Sensitive to Outliers

Scikit-Learn

Scikit-Learn Core Components

1. **Datasets**
2. **preprocessing transformers**
3. **Estimators**
4. **Pipelines**
5. **Model Evaluation**
6. **Hyperparameter Tuning**

Scikit-Learn

Datasets

Scikit-learn provides access to several standard datasets like Iris, digits, and Boston housing.

```
# Import necessary libraries
from sklearn.datasets import load_iris
import pandas as pd

# Load the Iris dataset
iris = load_iris()

# Convert the dataset to a Pandas DataFrame
iris_df = pd.DataFrame(data=iris.data, columns=iris.feature_names)

# Add the target variable to the DataFrame
iris_df['species'] = iris.target

# Display the first few rows of the DataFrame
print(iris_df.head())
```

Scikit-Learn

preprocessing transformers

Transformers in scikit-learn are used for data preprocessing, implementing both *fit* and *transform* methods to prepare data for modeling.

- **StandardScaler:** Standardizes features by removing the mean and scaling to unit variance.
- **MinMaxScaler:** Scales features to a specified range.
- **PCA** (Principal Component Analysis): Reduces dimensionality.
- **OneHotEncoder:** Encodes categorical features as a one-hot numeric array.

Scikit-Learn

preprocessing transformers

```
from sklearn.preprocessing import StandardScaler
import numpy as np

# Sample data
data = np.array([[1, 2], [3, 4], [5, 6]])

# Initialize the StandardScaler
scaler = StandardScaler()

# Fit the scaler to the data
scaler.fit(data)

# Transform the data
scaled_data = scaler.transform(data)

print(scaled_data)
```

What will be the output?

Scikit-Learn

Estimators

Estimators are used for modeling and prediction tasks.

They provide implementations for regression, classification, and clustering models.

Core Methods of Estimators:

- ***fit***: Trains the model on the provided data.
- ***predict***: Generates predictions based on the trained model.
- ***score***: Evaluates model performance on test data.

Common Scikit-Learn Estimators:

- ***LinearRegression***: For regression tasks, predicting continuous values.
- ***LogisticRegression***: For binary and multi-class classification.
- ***SVM***: Used for classification and regression with margin-based learning.
- ***RandomForestClassifier***: An ensemble model for classification tasks.

Scikit-Learn

Estimators

```
from sklearn.linear_model import LinearRegression
import numpy as np

# Sample data
X = np.array([[1, 1], [1, 2], [2, 2], [2, 3]])
y = np.dot(X, np.array([1, 2])) + 3

# Initialize the LinearRegression model
model = LinearRegression()

# Fit the model to the data
model.fit(X, y)

# Make predictions
predictions = model.predict(X)

print(predictions)
```

Scikit-Learn

Pipelines

Pipelines are used to chain multiple steps together, such as data transformation and model fitting, ensuring that the entire workflow is applied consistently.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

# Sample data
X = [[0, 0], [1, 1]]
y = [0, 1]

# Create a pipeline
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('logistic', LogisticRegression())
])

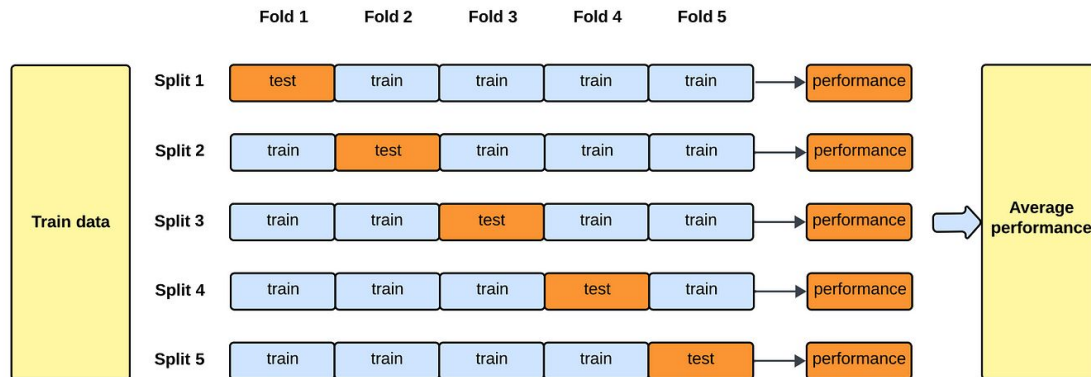
# Fit the pipeline to the data
pipeline.fit(X, y)

# Make predictions
predictions = pipeline.predict(X)

print(predictions)
```

Model evaluation involves assessing the performance of an estimator using metrics like: accuracy, precision, recall, and F1 score.

Cross-validation is a common technique for model evaluation, which ensures that the model's performance is validated across multiple subsets of the data.



Scikit-Learn

Model Evaluation

```
from sklearn.model_selection import cross_val_score
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_iris

# Load the iris dataset
iris = load_iris()
X, y = iris.data, iris.target

# Initialize the RandomForestClassifier
clf = RandomForestClassifier()

# Perform cross-validation
scores = cross_val_score(clf, X, y, cv=5)

print(scores)
```

Hyperparameter tuning involves optimizing the hyperparameters of an estimator to improve its performance. This is done using methods like *GridSearchCV* and *RandomizedSearchCV*.

GridSearchCV: Performs an exhaustive search over a specified parameter grid.

RandomizedSearchCV: Performs a random search over specified parameter values, which can be more efficient.

```
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier

param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [None, 10, 20, 30]
}

grid_search = GridSearchCV(RandomForestClassifier(), param_grid, cv=5)
grid_search.fit(X_train, y_train)
print(grid_search.best_params_)
print(grid_search.best_score_)
```